

Album Burst — Product Requirements

Version 1.1 · Design & Build Reference

1. Project Overview

Album Burst is a web application that generates organic, data-driven visualizations of music albums. Each song in an album is represented as a hand-drawn burst/flower shape whose visual properties are encoded from musical data: duration, BPM, key, accidental, and mode. Shapes are arranged in a circle-packed layout and colored using a chromatic key ramp.

The primary use case is personal and exploratory — users upload their own album data via CSV and receive a downloadable square visualization poster. A set of preloaded albums is provided so users can explore the tool before uploading their own data.

2. Data Model

2.1 Song Data Schema

Each song is represented by the following fields:

- track — Integer. Track number (used for layout order).
- name — String. Song title.
- duration — Integer. Duration in seconds.
- bpm — Integer. Beats per minute.
- key — String. Root note letter only: C, D, E, F, G, A, or B.
- accidental — String. One of: natural, sharp, flat.
- mode — String. One of: major, minor.

2.2 Visual Encoding

- **Duration (square root scale, range 36–130px diameter).**Size →
- **BPM ÷ 15, minimum 3 petals.**Petal count →
- **Accidental. Natural = rounded egg/teardrop. Sharp = narrow spike triangle. Flat = rectangular plank.**Petal shape →
- **Mode. Major = irregular hand-drawn oval. Minor = irregular hand-drawn triangle (randomly rotated).**Center cutout shape →

- **Key letter (A–B), mapped to a 12-step chromatic ramp (C=dark end, B=light end). Accidentals share the color of their root letter. Interpolated using `d3.interpolateHcl` for perceptual uniformity.**Color →

2.3 Shape Library

Each petal type uses a set of 5 unique hand-drawn SVG paths. Petals cycle through the library and are rotated into position around the flower center. Each petal gets a slight random rotation jitter ($\pm 6^\circ$ for most types, $\pm 3^\circ$ for sharp petals at low counts) seeded per flower. The overall flower receives a random global rotation.

Center cutouts (major oval and minor triangle) are hand-drawn paths stored as normalized SVG coordinates. Both receive a random rotation per flower instance.

3. Layout Engine

Songs are arranged using a D3 force simulation:

- Initial positions seeded using a golden angle spiral to prevent grid structure.
- `forceCollide`: radius = 80% of shape radius, strength 1.0, 4 iterations.
- `forceCenter`: strength 0.08.
- `forceX/Y` toward spiral seed positions: strength 0.04.
- 800 ticks, run synchronously before render.
- Slight overlap between shapes is intentional.

Canvas is square. A dynamic `viewBox` is computed from the bounding box of all placed shapes plus padding, scaled to a fixed square display area.

4. Color & Palette System

Palettes are defined as simple objects in a single `PALETTES` array in `palettes.js` (or equivalent config file). This is the only file that needs to be edited to add, remove, or modify palettes. Each palette has:

- `id` — unique string identifier
- `name` — display name shown in the palette picker
- `bg` — hex background color (also used for center cutout 'hole' color)
- `colorStart` — hex color for the C end of the chromatic ramp (darker/lower end)
- `colorEnd` — hex color for the B end of the chromatic ramp (lighter/upper end)

Example palette definition:

```
{ id: 'deep-navy', name: 'Deep Navy',  
  bg: '#001B2E', colorStart: '#294C60', colorEnd: '#ADB6C4' }
```

The color ramp uses `d3.scaleSequential` with `d3.interpolateHcl` for perceptual uniformity. The 12 chromatic steps map C, C#, D, D# ... B. Accidentals (sharp/flat) share the root letter color.

Initial palette set (5 palettes — subject to change, edit `palettes.js` freely):

- Deep Navy — bg #001B2E, #294C60 → #ADB6C4
- Crimson — bg #6B2737, #E08E45 → #F8F4A6
- Gray Olive — bg #5E5D5C, #BCBD8B → #EFF1ED
- Dark Teal — bg #00272B, #FF6663 → #E0FF4F
- Near Black — bg #030301, #8D6346 → #DDF45B

5. Screens & UI

5.1 Home Screen

A vertically scrolling feed of preloaded album visualizations (~5 albums at launch). Each list item shows the square visualization, album title, and artist name. Tapping navigates directly to the Visualization Screen.

Fixed bottom bar: CREATE button (left, prominent) opens the Upload Data overlay. Short explanatory text to the right.

5.2 Upload Data Overlay

Bottom sheet over dimmed home screen. Contains:

- Title 'Upload Your Album Data' + X close button.
- Short instructional text.
- Copyable LLM prompt block with COPY button. Expand/collapse toggle — collapsed by default.
- Horizontal divider.
- UPLOAD CSV button with spreadsheet icon.

On successful upload: overlay closes, navigates to Visualization Screen. CSV is validated for required columns and value ranges before navigating.

5.3 Visualization Screen

Main view after selecting a preloaded album or uploading data.

- Top left: Album name + artist name (two lines, small text).
- Top right: ⓘ Info button — opens Key/Legend overlay.
- Center: Square visualization with breathing room above and below.
- Bottom bar: Back arrow (left), Palette icon (center), Download icon (right).

Song Card Carousel (Yelp-style):

- Tapping a flower reveals a song card at the bottom of the screen.
- Cards are swipeable horizontally to navigate through all songs in track order.
- Active flower is highlighted in the visualization while its card is shown.
- Dismiss by tapping outside, swiping down, or close button.
- Each card shows: song name, duration (m:ss), BPM, key + accidental, major/minor.

5.4 Palette Overlay

Bottom sheet. Title 'Pick Your Palette' + X close. 2-column scrollable grid of palette thumbnails, each showing a mini visualization preview with palette name below. Tapping applies immediately and closes. Active palette shown with selected state.

5.5 Key / Legend Overlay — v1 Placeholder

Triggered by ⓘ button. For v1, implement as a simple bottom sheet with text-only descriptions of the encoding system. Full visual design to be completed in a later iteration.

Placeholder content:

- SIZE = how long the song is
- PETALS = tempo (BPM)
- PETAL SHAPE = accidental — rounded (natural), spiky (sharp), rectangular (flat)
- CENTER SHAPE = mode — oval hole (major), triangle hole (minor)
- COLOR = musical key, C through B

Mark the component clearly with a TODO comment for visual redesign.

5.6 Song Card — v1 Placeholder

For v1, implement as a simple bottom card with a clean list of song data fields. Full visual design to be completed in a later iteration. Swipe/tap-away dismiss behavior should be fully implemented even in v1.

Card fields: Song name (prominent), Duration, BPM, Key + accidental, Major/Minor.

Mark the component clearly with a TODO comment for visual redesign.

5.7 Download Overlay — Planned

Options: title on/off, square or vertical format. Exports high-resolution PNG from SVG. Design and implementation TBD.

6. CSV Format

Required columns (headers case-insensitive):

- track — integer
 - name — string
 - duration — integer (seconds)
 - bpm — integer
 - key — one of: C, D, E, F, G, A, B
 - accidental — one of: natural, sharp, flat
 - mode — one of: major, minor
-

7. LLM Prompt

The following prompt is displayed in the Upload Data overlay for users to copy into their LLM of choice. It instructs the LLM to look up and format album data as a CSV.

— PROMPT START —

I need you to look up music data for an album and format it as a CSV file I can download.

Album: [ALBUM NAME] by [ARTIST NAME]

Please look up the following data for every track on the album using a source like Musicstax, Tunebat, or GetSongBPM, and create a CSV with these exact column headers:

track, name, duration, bpm, key, accidental, mode

Rules for each field:

- track — track number as an integer (1, 2, 3...)
- name — song title as a string
- duration — song length in seconds as an integer (e.g. 3:45 = 225)
- bpm — beats per minute as an integer. If two sources disagree, prefer the higher value. If the track has no clear beat, use your best estimate.
- key — the root note letter only, one of: C, D, E, F, G, A, B (no sharps or flats here)
- accidental — one of: natural, sharp, flat (e.g. F# = key F, accidental sharp / Bb = key B, accidental flat / C major = key C, accidental natural)
- mode — one of: major or minor

Please include every track on the standard album (no bonus tracks or deluxe editions unless specified). Output only the CSV data with no explanation, just the header row followed by one row per track.

— PROMPT END —

Note: The placeholder text [ALBUM NAME] and [ARTIST NAME] should remain editable in the prompt block so users can fill them in before copying.

8. Technical Stack

- Framework: React (Vite)
- Visualization: D3 (force simulation, color scales)
- SVG rendering: Inline React SVG components
- CSV parsing: PapaParse
- Styling: Tailwind CSS
- Hosting: Vercel (deploy from GitHub)
- No backend required for v1 — all processing client-side

9. Preloaded Albums

- Boards of Canada — Music Has the Right to Children (18 tracks) — default palette: Deep Navy
 - Frank Ocean — Blonde (17 tracks) — default palette: Crimson
 - Taylor Swift — Speak Now (14 tracks) — default palette: Gray Olive
 - Sex Pistols — Never Mind the Bollocks (12 tracks) — default palette: Dark Teal
 - Radiohead — In Rainbows (10 tracks) — default palette: Near Black
-

10. Open Items

- Key/Legend overlay visual design — placeholder in v1, full design TBD.
 - Song card visual design — placeholder in v1, full design TBD.
 - Download/export implementation — high-res PNG from SVG, TBD.
 - Cut-paper edge effect on petal paths — discussed, low priority for v1.
 - Spotify API integration — blocked by endpoint deprecation (post Nov 2024). Revisit if access restored.
 - Desktop layout — current designs mobile-first. Desktop TBD.
 - Additional palettes — system is designed to be extensible via palettes.js.
 - LLM prompt editable fields — [ALBUM NAME] and [ARTIST NAME] should be inline-editable in the UI before copying.
-

11. Reference Implementation

The core visualization is already prototyped and working. Claude Code should use these files as the direct source of implementation — do not rebuild the visualization from scratch.

11.1 File Structure (in repo)

- docs/sketches/ — UI wireframe sketches (4 screens, see each for layout reference)
- prototypes/album-boc.jsx — Full working visualization component, Boards of Canada data, Deep Navy palette
- prototypes/album-blonde.jsx — Frank Ocean Blonde, Crimson palette
- prototypes/album-speaknow.jsx — Taylor Swift Speak Now, Gray Olive palette
- prototypes/album-pistols.jsx — Sex Pistols NMTB, Dark Teal palette
- prototypes/album-rainbows.jsx — Radiohead In Rainbows, Near Black palette

11.2 What Is Stable — Do Not Change

- All SVG petal path data (NATURAL_PETALS, SHARP_PETALS, FLAT_PETALS arrays) — these are hand-drawn and cannot be regenerated algorithmically.
- MAJOR_PATH and MINOR_PATH — hand-drawn center cutout shapes.
- parseBBox() and seededRand() utility functions.
- The Flower component rendering logic — petal placement, width scaling, jitter, cutout rendering.
- The D3 force simulation layout — golden angle spiral seed, force parameters, 800 ticks.
- The HCL color interpolation approach (d3.interpolateHcl).
- The NOTE_ORDER chromatic scale mapping (C through B, 12 steps).

11.3 What Should Be Refactored for the App

- Song data — move from inline constants into a structured data file (e.g. src/data/albums.js) with one exported array per album.
- Palette definitions — extract into src/data/palettes.js as a named array. This is the only file that should need editing to change or add palettes.
- The Flower and visualization components — split into separate component files (e.g. src/components/Flower.jsx, src/components/Visualization.jsx).
- The petal path libraries — extract into src/data/petalPaths.js to keep component files clean.
- BPM divisor (currently 15) — extract as a named constant so it can be tuned easily.
- Size scale range (currently 36–130px) — extract as named constants.

11.4 Key Implementation Notes

- The center cutout color must always match the background color (BG constant) so it reads as a hole, not a filled shape.
- Sharp petals at 5 or fewer use reduced jitter ($\pm 3^\circ$) to prevent corner gaps at the base. All other petal types use $\pm 6^\circ$.
- Sharp petals also extend their base 6% deeper into center (baseY adjustment) to cover gaps from rotation.
- Natural petals use a smaller center patch radius (9% of size) vs flat/sharp (14%) due to how the petal bases overlap.
- Width scaling uses $\text{Math.sqrt}(9 / \text{petalCount})$ with a cap of 1.15x for sharp petals to prevent over-splaying at low counts.
- The viewBox is computed dynamically from the actual bounding box of placed nodes — do not use a fixed viewBox.